

Draw simple figure



Contents

- 1 import
- 2 0으로 채운 행렬 하나~ 근데 shape이...
- 3 RGB가 다 0이면~ 까망색
- 4 사각형 그리는 함수
- 5 사각형 그려보기
- 6 이번에는 파랑색 박스 추가
- 7 원 그리는 함수
- 8 원 그리기~
- 9 thickness를 -1로 하면
- 10 직선 만드는 함수
- 11 직선 만들기
- 12 글자 넣는 함수
- 13 글자 넣기
- 14 글자 넣기 결과
- 15 새로 시작하면서, 네 꼭지점을 저장하자
- 16 shape은 이렇게 생겼지
- 17 살짝 reshape을~
- 18 이게 뭐냐면~
- 19 다각형 그리는 함수
- 20 다각형 그리기

21 한번더 선그리기

22 사각형 그리기

23 원과 다각형

Draw simple figure

Exported from Confluence · <https://pinkwink.atlassian.net>

import

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Python

0으로 채운 행렬 하나~ 근데 shape이...

```
blank_img = np.zeros((512, 512, 3), dtype=np.uint8)
blank_img.shape
```

✓ 0.0s

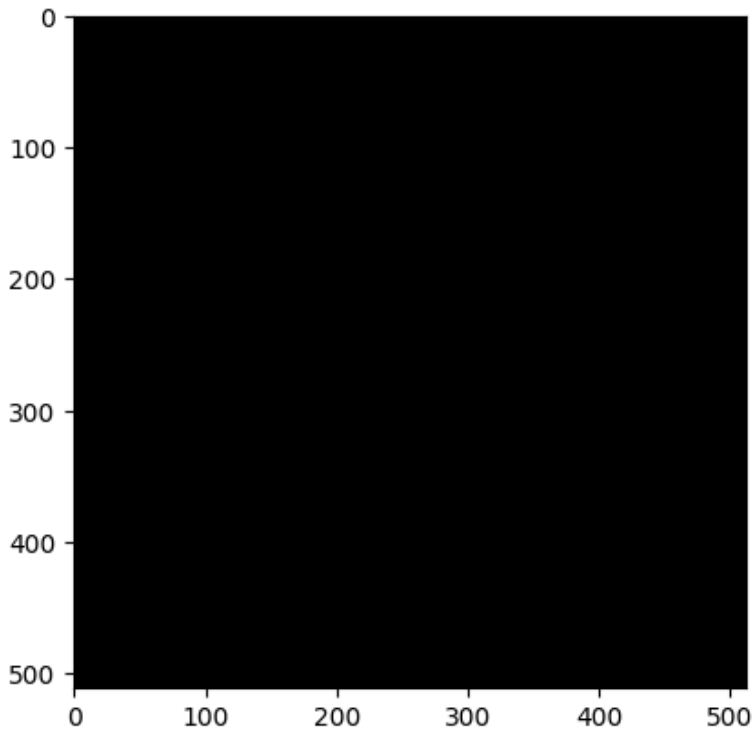
Python

(512, 512, 3)

RGB가 다 0이면~ 까망색

```
plt.imshow(blank_img);
```

Python



사각형 그리는 함수

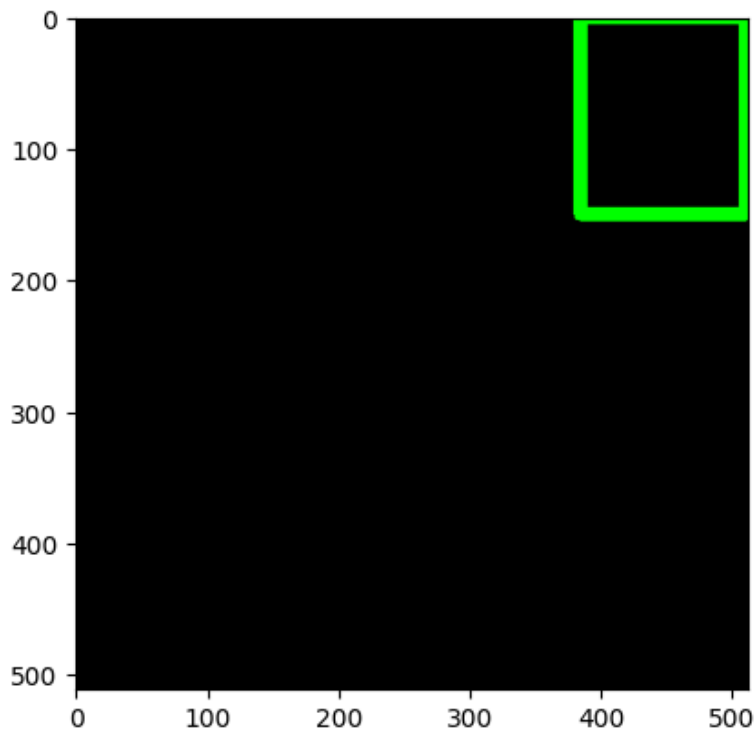
```
cv2.rectangle(img, pt1, pt2, color, thickness=None, lineType=None,
               shift=None) -> img
cv2.rectangle(img, rec, color, thickness=None, lineType=None,
               shift=None) -> img
```

- img: 그림을 그릴 영상
- pt1, pt2: 사각형의 두 꼭지점 좌표. (x, y) 튜플.
- rec: 사각형 위치 정보. (x, y, w, h) 튜플.
- color: 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- thickness: 선 두께. 기본값은 1. 음수(-1)를 지정하면 내부를 채움.
- lineType: 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택. 기본값은 cv2.LINE_8
- shift: 그리기 좌표 값의 축소 비율. 기본값은 0.

사각형 그려보기

```
cv2.rectangle(  
    blank_img, pt1=(384, 0), pt2=(510, 150), color=(0, 255, 0), thickness=10  
)  
  
plt.imshow(blank_img)
```

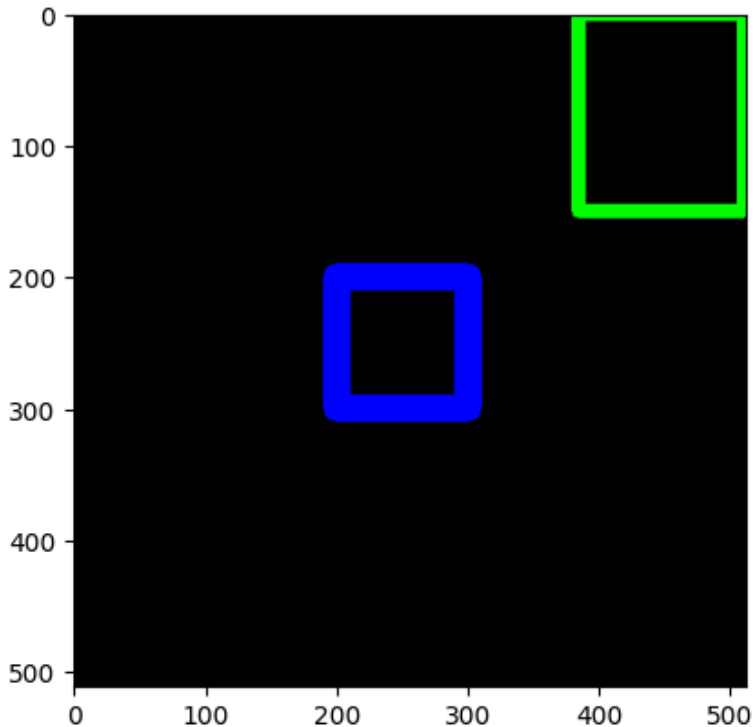
Python



이번에는 파랑색 박스 추가

```
cv2.rectangle(
|   blank_img, pt1=(200, 200), pt2=(300, 300), color=(0, 0, 255), thickness=20
| )
plt.imshow(blank_img)
```

Python



원 그리는 함수

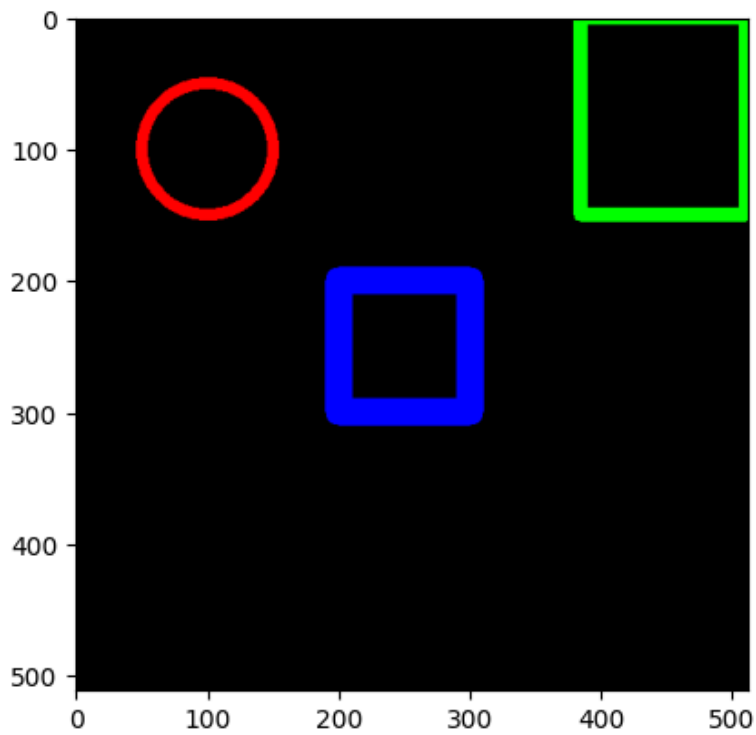
```
cv2.circle(img, center, radius, color, thickness=None, lineType=None,
            shift=None) -> img
```

- img: 그림을 그릴 영상
- center: 원의 중심 좌표. (x, y) 튜플.
- radius: 원의 반지름
- color: 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- thickness: 선 두께. 기본값은 1. 음수(-1)를 지정하면 내부를 채움.
- lineType: 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택. 기본값은 cv2.LINE_8
- shift: 그리기 좌표 값의 축소 비율. 기본값은 0.

원 그리기~

```
cv2.circle(  
|   blank_img, center=(100, 100), radius=50, color=(255, 0, 0), thickness=8  
|   )  
  
plt.imshow(blank_img)
```

Python

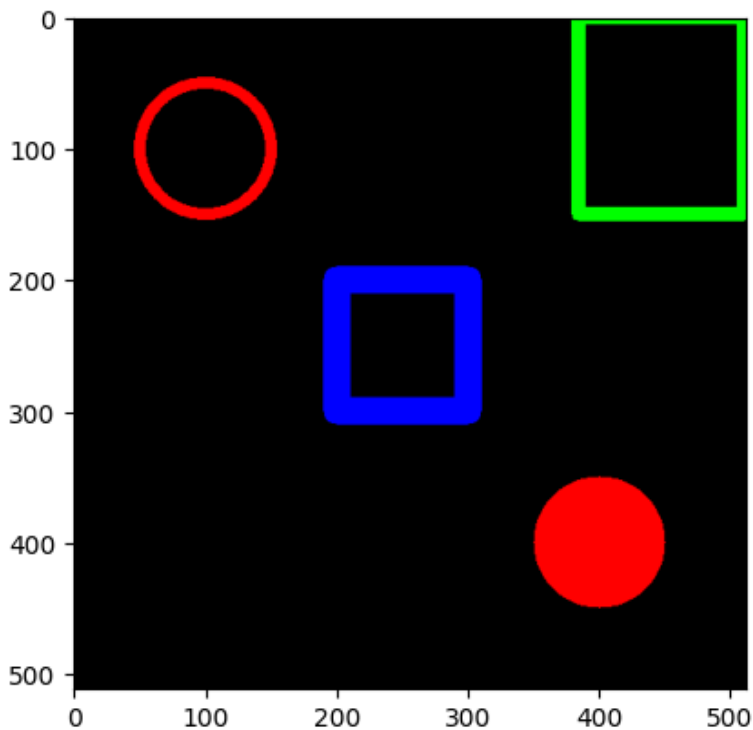


thickness를 -1로 하면


```
cv2.circle(
    blank_img, center=(400, 400), radius=50, color=(255, 0, 0), thickness=-1
)

plt.imshow(blank_img)
```

Python



직선 만드는 함수

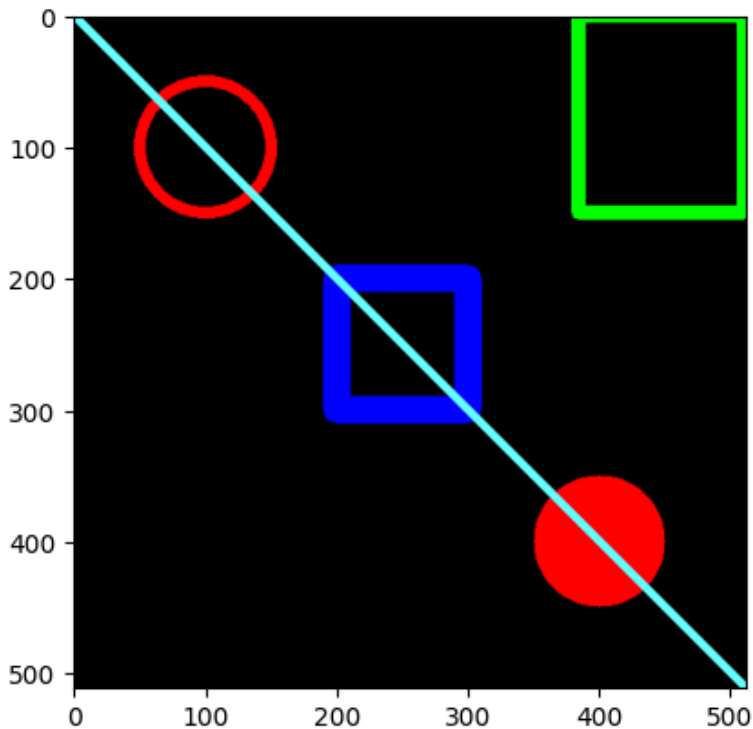
```
cv2.line(img, pt1, pt2, color, thickness=None, lineType=None,
        shift=None) -> img
```

- img: 그림을 그릴 영상
- pt1, pt2: 직선의 시작점과 끝점. (x, y) 튜플.
- color: 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- thickness: 선 두께. 기본값은 1.
- lineType: 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택. 기본값은 cv2.LINE_8
- shift: 그리기 좌표 값의 축소 비율. 기본값은 0.

직선 만들기

```
cv2.line(blank_img, pt1=(0, 0), pt2=(512, 512), color=(102, 255, 255), thickness=5)
plt.imshow(blank_img)
```

Python



글자 넣는 함수

```
cv2.putText(img, text, org, fontFace, fontScale, color, thickness=None,
            lineType=None, bottomLeftOrigin=None) -> img
```

- **img:** 그림을 그릴 영상
- **text:** 출력할 문자열
- **org:** 영상에서 문자열을 출력할 위치의 좌측 하단 좌표. (x, y) 튜플.
- **fontFace:** 폰트 종류. `cv2.FONT_HERSHEY_`로 시작하는 상수 중 선택
- **fontScale:** 폰트 크기 확대/축소 비율
- **color:** 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- **thickness:** 선 두께. 기본값은 1. 음수(-1)를 지정하면 내부를 채움.
- **lineType:** 선 타입. `cv2.LINE_4`, `cv2.LINE_8`, `cv2.LINE_AA` 중 선택.
- **bottomLeftOrigin:** True이면 영상의 좌측 하단을 원점으로 간주. 기본값은 False.

글자 넣기

```
font = cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(
    blank_img,
    text="Hello",
    org=(10, 500),
    fontFace=font,
    fontScale=4,
    color=(255, 255, 255),
    thickness=3,
    lineType=cv2.LINE_AA,
);
```

✓ 0.0s

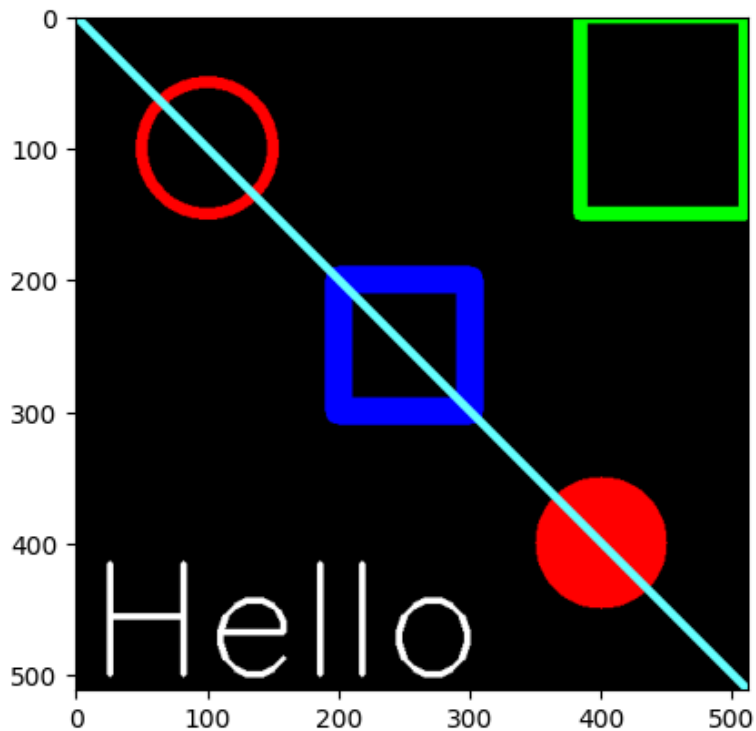
Python

글자 넣기 결과

```
plt.imshow(blank_img);
```

✓ 0.0s

Python



새로 시작하면서, 네 꼭지점을 저장하자

```
blank_img = np.zeros((512, 512, 3), dtype=np.int32)
vertices = np.array([[100, 300], [200, 200], [400, 300], [200, 400]], dtype=np.int32)
vertices
```

✓ 0.0s

Python

```
array([[100, 300],
       [200, 200],
       [400, 300],
       [200, 400]], dtype=int32)
```

shape은 이렇게 생겼지

```
vertices.shape
```

✓ 0.0s

Python

```
(4, 2)
```

살짝 reshape을~

```
pts = vertices.reshape((-1, 1, 2))
pts.shape
```

✓ 0.0s

Python

```
(4, 1, 2)
```

이게 뭐냐면~

```
vertices
```

✓ 0.0s

Python

```
array([[100, 300],
       [200, 200],
       [400, 300],
       [200, 400]], dtype=int32)
```

```
pts
```

✓ 0.0s

Python

```
array([[[100, 300]],
       [[200, 200]],
       [[400, 300]],
       [[200, 400]]], dtype=int32)
```

다각형 그리는 함수

```
cv2.polylines(img, pts, isClosed, color, thickness=None, lineType=None,
               shift=None) -> img
```

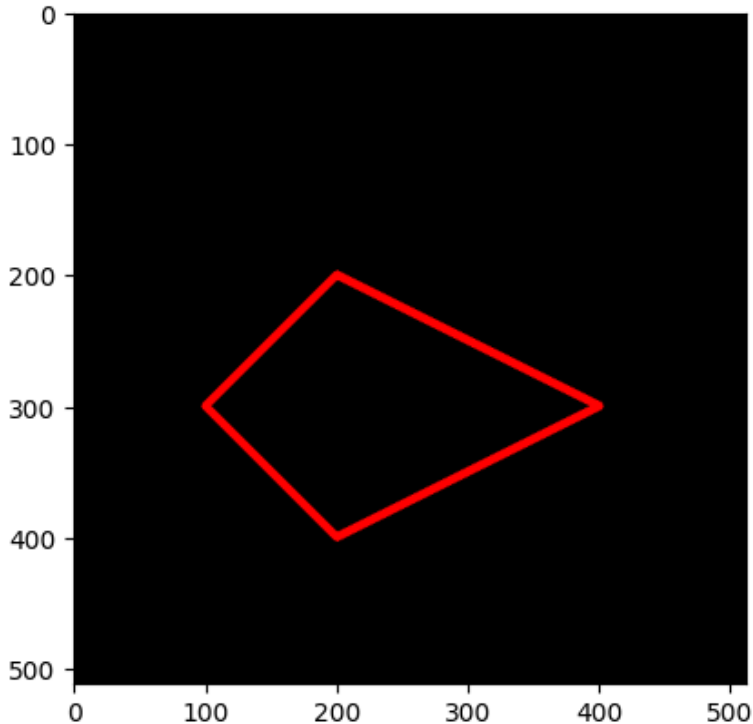
- img: 그림을 그릴 영상
- pts: 다각형 외곽 점들의 좌표 배열. [numpy.ndarray의 리스트](#).
(e.g.) `[np.array([[10, 10], [50, 50], [10, 50]], dtype=np.int32)]`
- isClosed: 폐곡선 여부. True 또는 False 지정.
- color: 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- thickness: 선 두께. 기본값은 1. 음수(-1)를 지정하면 내부를 채움.
- lineType: 선 타입. `cv2.LINE_4`, `cv2.LINE_8`, `cv2.LINE_AA` 중 선택.
기본값은 `cv2.LINE_8`
- shift: 그리기 좌표 값의 축소 비율. 기본값은 0.

다각형 그리기

```
cv2.polylines(blank_img, [pts], isClosed=True, color=(255, 0, 0), thickness=5)
plt.imshow(blank_img);
```

✓ 0.0s

Python



한번더 선그리기

```
import numpy as np

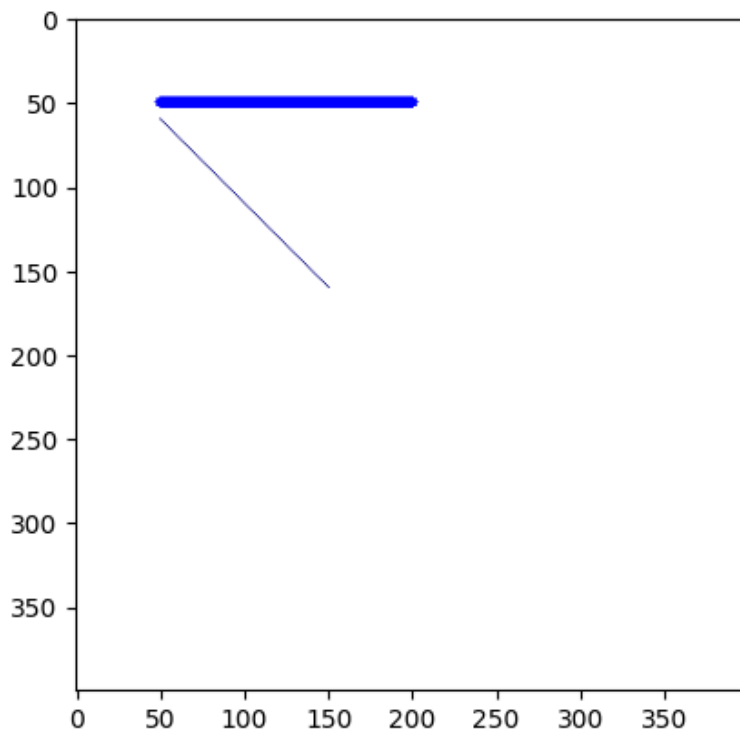
img = np.full((400, 400, 3), 255, np.uint8)

cv2.line(img, (50, 50), (200, 50), (0, 0, 255), 5)
cv2.line(img, (50, 60), (150, 160), (0, 0, 128))

plt.imshow(img);
```

✓ 0.0s

Python



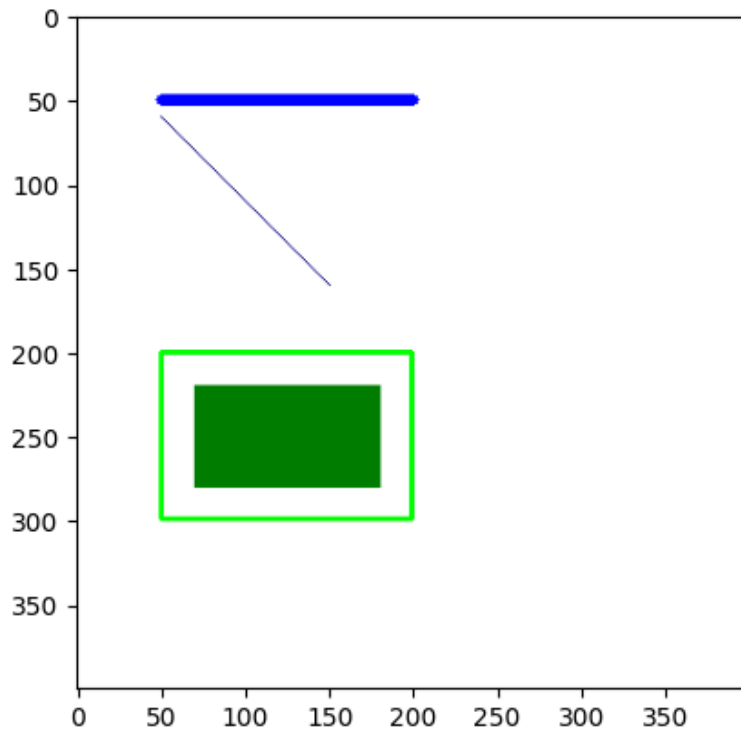
사각형 그리기

```
cv2.rectangle(img, (50, 200, 150, 100), (0, 255, 0), 2)
cv2.rectangle(img, (70, 220), (180, 280), (0, 128, 0), -1)

plt.imshow(img);
```

✓ 0.0s

Python



원과 다각형

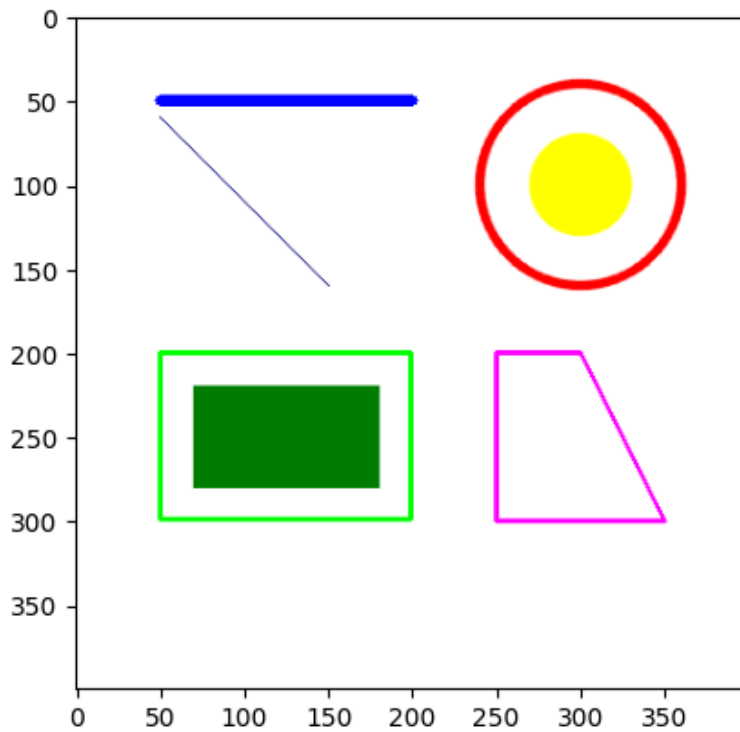
```
cv2.circle(img, (300, 100), 30, (255, 255, 0), -1, cv2.LINE_AA)
cv2.circle(img, (300, 100), 60, (255, 0, 0), 3, cv2.LINE_AA)

pts = np.array([[250, 200], [300, 200], [350, 300], [250, 300]])
cv2.polylines(img, [pts], True, (255, 0, 255), 2)

plt.imshow(img);
```

✓ 0.0s

Python





PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio